

BRC Documentation: Introduction to Hazel-HPC

Mery Touceda-Suárez

2026-04-23

Table of contents

02. Running jobs on Hazel-HPC	1
Interactive Jobs	1
Anatomy of a Job Script (testing changes)	2
Job Monitoring and Management	7
Queues on Hazel (from HazelWiki)	8
Standard Output and Standard Error	9
Common Errors and How to Fix Them	9
Debugging Tips	11

02. Running jobs on Hazel-HPC

What is a job

An HPC job is the unit of work that a user submits to the cluster’s Workload Manager (or job scheduler) for execution. It’s essentially your program, simulation code, or analysis script, bundled with all the necessary instructions for the system to run it. These instructions are typically contained within a job script (or submission script), which includes vital directives for the scheduler. By encapsulating the executable code, resource requirements, and runtime parameters, the job acts as a complete, self-contained package that the scheduler can manage and allocate hardware for, ensuring efficient use of the shared cluster resources.

LSF Job Scheduler

Hazel uses the LSF (Load Sharing Facility) job scheduler, currently known as IBM Spectrum LSF. Users submit jobs using the `bsub` command and include specific directives (e.g., `#BSUB -n 16`) to communicate their resource needs to the scheduler. LSF then handles the entire lifecycle of the job—from submission to completion, including fault tolerance and output logging.

Interactive Jobs

For quick testing, debugging, or running applications that require a graphical interface (GUI), you must request a temporary allocation on a compute node using an interactive job.

```
$ bsub -Is -n 1 -W 10 bash
```

Option	Meaning
<code>-I</code>	Requests an interactive session.
<code>-s</code>	Sends a signal to the job shell upon termination.
<code>-n 1</code>	Requests 1 CPU core .
<code>-W 10</code>	Requests 10 minutes of wall-clock time.

Option	Meaning
bash	The shell to run on the allocated node.

Requesting multiple cores on Interactive

It is possible to request multiple cores for an interactive node. For example, to request an interactive session using **4 cores** (-n 4), with **all cores on the same node** (-R "span[hosts=1]"), and **10 minutes** of wall clock time (-W 10) use:

```
bsub -Is -n 4 -R "span[hosts=1]" -W 10 bash
```

Anatomy of a Job Script (testing changes)

Job scheduling software. LSF (Load sharing facility) Job contains: the script you want to run and information about the resources you need to run it.

0. Example of a job script:

```
#!/bin/bash
# =====
# Hello World python Job Script for Hazel HPC
# =====
# This job script submits a python program to print "Hello World"

# -----
# Request resources here
# -----
#BSUB -J hello                # job name
#BSUB -n 2                    # number of CPUs required per task
#BSUB -q shared_memory        # the queue to run on
#BSUB -R "span[hosts=1]"      # number of hosts to spread the jobs across, 1 host used here
#BSUB -R "rusage[mem=4GB]"    # required total memory for the job
#BSUB -o "./output.%J.log"    # standard output file (%J is job name)
#BSUB -e "./error.%J.log"     # standard error file (%J is job ID)
#BSUB -W 10:00                # time to run

# -----
# Load modules here
# -----

module load python

# -----
# Execute commands here
# -----

python hello.py
```

1. Request resources block

The “Request Resources Block” is the most critical section, as it uses LSF directives (lines starting with `#BSUB`) to tell the Workload Manager exactly how much computing power and time your job needs.

Directive	Command	Description
Job Name	<code>#BSUB -J hello</code>	Sets the job name to hello . This is used to identify the job in the queue and in output files.
CPU Cores	<code>#BSUB -n 2</code>	Requests 2 CPU cores per task. For a parallel job, this is the number of cores dedicated to that single task.
Queue	<code>#BSUB -q shared_memory</code>	Specifies the queue (or partition) where the job should run
Host Span	<code>#BSUB -R "span[hosts=1]"</code>	Requests that all required resources (the 2 cores) for the task must reside on a single computing host (node).
Memory	<code>#BSUB -R "rusage[mem=4GB]"</code>	Requests 4 GB of total memory (RAM) for the job task. The job will be scheduled only on a node with at least this much available memory.
Output File	<code>#BSUB -o "./output.%J.log"</code>	Directs standard output (stdout) to this file.
Error File	<code>#BSUB -e "./error.%J.log"</code>	Directs standard error (stderr) to this file.
Wall Time	<code>#BSUB -W 10:00</code>	Sets the maximum wall-clock time (actual time elapsed) to 10 minutes . If the job runs longer, LSF will terminate it.

2. Load modules block

This block is crucial for setting up the software environment your specific job requires. HPC systems use a tool called Modules to manage different versions of software, compilers, and libraries without conflicts.

This section uses the `module load` command to prepare the job’s environment. This avoids conflicts by allowing different users and different jobs to use specific, isolated versions of software on the shared cluster.

Sometimes the **software you need won’t be available as a pre-installed module** in the HPC, in that case you will need to **install it yourself** before you run it. We recommend trying to use **containers** first, and if that is not possible try to install the software using an virtual environment manager like **conda**.

There will be more information on this in later documentation but in case you installed your software with any of these options **this is how you would load/activate them in your job script**:

Command	Example	Description
Setup Option 1	<code>module load conda</code>	Loads the Conda module, which manages virtual environments. This makes the <code>conda</code> command available for use.
Activation	<code>conda activate my_env</code>	<i>If using Conda</i> , this line activates the specific virtual environment containing your job's required tools.
Setup Option 2	<code>module load apptainer</code>	Loads the Apptainer module for running containerized applications. Containers package software with all its dependencies.

Key Principle: Always load the modules your job needs *before* trying to run the main command.

3. Execute commands block

This is the main body of the script where your actual scientific or computational work takes place. These are standard shell commands that will be executed sequentially on the allocated compute node(s). This is where you place the executable commands that perform the job's core task. It's often wrapped with simple `echo` commands to log key steps, helping with debugging and monitoring.

The example above has a very simplified execution compared to what one would normally make or encounter in bioinformatics. This is a more realistic example of execution of the software **fastqc**:

```
# -----
# Load modules here
# -----

module load apptainer

# -----
# Execute commands here
# -----

# Print job start information
echo "=====
echo "Job started: $(date)"
echo "Job ID: $LSB_JOBID"
echo "Running on host: $(hostname)"
echo "Working directory: $(pwd)"
echo "=====

FASTQC_SIF="/rs1/shares/brc/admin/containers/images/quay.io_biocontainers_fastqc:0.12.1--hdfd78af_0.sif
IN_DIR="/rs1/shares/brc/trainings/hazel_hpc/data"
SAMPLE="sample_001"

mkdir -p /path/to/working/dir/fastqc_results
OUT_DIR="/path/to/working/dir/fastqc_results"

# -----
```

```

# Validate inputs
# -----

# Check if container exists
if [ ! -f "${FASTQC_SIF}" ]; then
    echo "ERROR: Container not found at ${FASTQC_SIF}"
    exit 1
fi

# Check if input directory exists
if [ ! -d "${IN_DIR}" ]; then
    echo "ERROR: Input directory not found at ${IN_DIR}"
    exit 1
fi

# -----
# Execute FastQC
# -----

echo "Running FastQC on sample: ${SAMPLE}"

apptainer exec $BIND ${FASTQC_SIF} fastqc \
    --threads $CPUS \
    --outdir ${OUT_DIR} \
    ${IN_DIR}/${SAMPLE}_*.fastq*

# Check if FastQC completed successfully
if [ $? -eq 0 ]; then
    echo "FastQC completed successfully"
else
    echo "ERROR: FastQC failed with exit code $?"
    exit 1
fi

# -----
# Job completion
# -----

echo "======"
echo "Job completed: $(date)"
echo "Results saved to: ${OUT_DIR}"
echo "======"

```

As you can see here we have included **examples of commonly used techniques in job scripting**:

- Variable creation and value setting
- Passing of job information to the standard output to help debugging and monitoring
- Creation of output directories
- Error checking Additionally, note that the the software **fastqc** is executed from an Apptainer container, not a module, so we also had to include paths to the container image, apart from input, output, etc.

Here is a detailed explanation of each line in case you would like to include any of these tools to your job scripts:

Command	Example	Description
Start Log	<pre>echo "Job started: \$(date)"echo "Job ID: \${LSB_JOBID}"echo "Running on host: \$(hostname)"echo "Working directory: \$(pwd)"</pre>	Logs the start time and environment details into your standard output (output.%.log) file: • <code>date</code> - records when the job started • <code>LSB_JOBID</code> - shows the unique job ID assigned by LSF • <code>hostname</code> - shows which compute node is running the job • <code>pwd</code> - prints current working directory
Set Variables	<pre>FASTQC_SIF="/rs1/shares/brc/Define/containers/images/quay.io/biocontainers-fastqc/fastqc:0.11.1" IN_DIR="/rs1/shares/brc/trajectories/ChazalIndic/data" SAMPLE="sample1"</pre>	Define variables for the task: • <code>FASTQC_SIF</code> - Container image location • <code>IN_DIR</code> - Input data directory • <code>OUT_DIR</code> - Output directory for results • <code>SAMPLE</code> - Sample identifier to process
Create Directories	<pre>mkdir -p /path/to/working/dir/fastqc_results</pre>	Creates the output directory if it doesn't already exist. The <code>-p</code> flag prevents errors if the directory already exists.
Validate Inputs	<pre>if [! -f "\${FASTQC_SIF}"]; then echo "ERROR: Container not found" exit 1fi</pre>	Checks that required files and directories exist before running the main task: • Verifies the container image exists • Confirms the input directory is accessible • Exits with error code if validation fails
Core Task	<pre>apptainer exec \${FASTQC_SIF} fastqc \ --threads 2 \ --outdir \${OUT_DIR} \ \${IN_DIR}/\${SAMPLE}_*.fastq.gz</pre>	Runs the main application using Apptainer to execute <code>fastqc</code> within the container: • <code>--threads 2</code> - uses 2 CPU cores (matches <code>#SUB -n 2</code>) • <code>--outdir</code> specifies where to save results • <code>fastqc</code> processes all FASTQ files matching the sample pattern
Error Checking	<pre>if [\$? -eq 0]; then echo "FastQC completed successfully"else echo "ERROR: FastQC failed" exit 1fi</pre>	Checks the exit status of the previous command: • <code>?</code> contains the exit code (0 = success) • Logs whether the task completed successfully • Exits with error code if the task failed
End Log	<pre>echo "Job completed: \$(date)"echo "Results saved to: \${OUT_DIR}"</pre>	Logs the end time and output location. This helps you: • Calculate actual runtime • Confirm the job completed • Know where to find results

Key Principle: Every command listed here runs on the allocated compute resources, not on the login node. This ensures your intensive computations don't overload the shared login environment.

Best Practices Shown:

- **Logging:** Track when and where your job runs
- **Variables:** Make paths easy to change and reuse
- **Validation:** Catch problems before wasting compute time
- **Error handling:** Know immediately if something goes wrong
- **Documentation:** Clear comments explain what each section does

! Important

All images should be in `/rs1/shares/brc/admin/containers/images`, if your tool is not there feel free to contact us and request it.

Job Monitoring and Management

These are some of the most relevant commands you will need to monitor your jobs:

Command Category	Command Syntax	Description
Submission	<code>bsub < yourjob.sh</code>	Submits the job script (<code>yourjob.sh</code>) to the LSF scheduler. The scheduler processes the <code>#BSUB</code> directives within the script.
Status Check	<code>bjobs</code>	Displays a list of all your jobs (running, pending, suspended, etc.) and their current status.
Detailed Status	<code>bjobs -l [JOBID]</code>	Provides a detailed report on a specific job, including the execution host, resource usage, and waiting reasons (if pending).
Termination	<code>bkill [JOBID]</code>	Terminates (kills) a running or pending job instantly. Use this if your job is faulty or you no longer need the results.
Resource Inquiry	<code>bhosts</code>	Shows the status of the compute nodes in the cluster, indicating which are available, busy, or otherwise unavailable.

1. Get job information with `bjobs`

The `bjobs` command is used to monitor the status of jobs after they are submitted to LSF. An LSF job status is usually in one of two states (STAT): `PEND` means that the job is queued and waiting for resources to become available and `RUN` means that the job is currently executing.

A typical `bjobs` output looks like:

```
[unityID@login01 ~]$ bjobs
JOBID  USER   STAT  QUEUE      FROM_HOST  EXEC_HOST  JOB_NAME  SUBMIT_TIME
948851  unityID RUN    standard  login01.hpc bc2j6      myjob1
```

Once you have the **JOBID** you can obtain more information about the job using:

```
bjobs -l JOBID
```

For jobs in a pending state, this will give information about why the job is pending. It is possible to make resource requests that are impossible to satisfy; jobs that have been pending for a long time may have made that type of request. It is possible that the chosen resources are currently being used, but it is also possible that the specified resources do not exist on the system. For example, LSF will not give an error message upon requesting a 64 core node with 500 GB of memory; it will simply wait until such a node is installed, leaving the job in a forever pending state.

Other job information commands:

- `bjobs -l`: get more detailed information on the jobs than `bjobs`
- `bjobs -lp -p3`: will include a list of reasons the job is pending, and it may include an estimate of when the job will start (*e.g.*, *Job will start no sooner than indicated time stamp*).
- `bjobs -u all | grep gpu`: find jobs running in a particular queue
- `bjobs -r -X -o "jobid queue cpu_used run_time avg_mem max_mem slots delimiter=',':"`: return a CSV formatted list of your jobs showing the job ID, queue, total CPU time, elapsed wall clock time, average memory utilized, maximum memory utilized, and the number of cores reserved

Note: Job priority is determined by several factors including fair share priority, queue priority, and time of submission.

2. Modify job requests with `bmod`

For example, to change the wall clock limit for a pending or running job, use

```
bmod -W [new time] [job ID].
```

3. Kill jobs with `bkill`

A pending job may be removed from the queue or a running job may be terminated by using the `bkill` command. The job ID is used to specify which job to remove.

```
bkill JOBID
```

To terminate all running and pending jobs, use `bkill 0`.

Queues on Hazel (from [HazelWiki](#))

To specify a queue, use `-q queue_name`.

Note

In general, **users should not specify a queue**. When no queue is specified, LSF will choose the most appropriate queue based on the number of cores and time requested from the set of default queues. The exceptions are partner queues and specialty queues, which are queues with special resources.

Default Queues

- debug
- serial
- short
- single_chassis
- standard
- long

Specialty Queues

- `shared_memory` - Nodes intended for running OpenMP or other shared memory executables particularly if have large memory requirements
- `gpu` - Nodes with attached NVIDIA GPUs
- `short_gpu` - Access to partner GPU nodes for up to 2 hour run time

The queues available to a user can be displayed by using `bqueues -u user_name`, and the properties of a queue can be displayed by using `bqueues -l queue_name`. For help interpreting the output of `bqueues`, see [this example](#)

```
(base) [mtouced@login03 ~]$ bqueues -u mtouced
```

QUEUE_NAME	PRIO	STATUS	MAX	JL/U	JL/P	JL/H	NJOBS	PEND	RUN	SUSP
debug	80	Open:Active	-	64	-	-	64	64	0	0
short	75	Open:Active	2560	1024	-	-	4229	2792	1237	0
shared_memory	70	Open:Active	-	256	-	-	2164	1086	1028	0
single_chassis	65	Open:Active	-	1024	-	-	6090	2759	2595	0
standard	64	Open:Active	-	1024	-	-	112	112	0	0
long	63	Open:Active	1536	512	-	-	1167	644	523	0
short_gpu	62	Open:Active	-	64	-	-	78	20	58	0
gpu	60	Open:Active	-	64	-	-	380	100	280	0
serial	30	Open:Active	2048	1024	-	-	342	63	279	0

bqueues output:

- NJOBS: number of total jobs in the queue
- RUN: Number of jobs actually running
- PEND: Number of jobs pending
- MAX: The maximum number of cores available. For some queues, like `gpu`, the MAX is not shown.

Other commands to get queue information

- `lshosts | grep gpu` or `bqueues -l gpu`: find which hosts have GPUs

Standard Output and Standard Error

When you submit a job to the HPC, your program's output needs to go somewhere. By default, programs write to two streams:

- **Standard Output (stdout)**: Normal program output and results
- **Standard Error (stderr)**: Error messages and warnings

Specifying Output Files in LSF

In your batch script, use these directives to control where output goes:

```
#BSUB -o stdout.%J    # Standard output file
#BSUB -e stderr.%J    # Standard error file
```

The `%J` is automatically replaced by your job ID when the job starts. For example, if your job ID is 948851, the files will be named: - `stdout.948851` - `stderr.948851`

You can customize the names:

```
#BSUB -o logs/myanalysis_out.%J
#BSUB -e logs/myanalysis_err.%J
```

Common Errors and How to Fix Them

1. File or Directory Missing

Error message in `stderr`:

```
/bin/bash: line 12: reads_R1.fastq: No such file or directory
```

or

```
cannot access 'data/samples.txt': No such file or directory
```

Causes:

- The file doesn't exist where you think it does
- You're using a relative path, but the job runs from a different directory
- Typo in the filename

Solutions:

- Use absolute paths: `/path/to/your/data/reads_R1.fastq`
- Verify the file exists before submitting: `ls -l reads_R1.fastq`
- Check your current directory: `pwd` (jobs run from the directory where you submit)
- Use `#BSUB -cwd /path/to/working/directory` to specify working directory

2. Out of Memory (OOM)

Error message in stderr:

```
slurmstepd: error: Detected 1 oom-kill event(s)
```

or

```
Killed  
java.lang.OutOfMemoryError: Java heap space
```

Cause: Your job exceeded the memory allocated or available on the node.

Solutions:

- Request more memory using `-R "rusage[mem=XXXX]"` (memory in MB per core)
- Check if your program has memory limit options
- Use a memory-efficient algorithm or subsample your data

3. Wall Time Exceeded

Error message in stderr:

```
TERM_RUNLIMIT: job killed after reaching LSF run time limit
```

Cause: Your job ran longer than the time specified with `-W`.

Solutions:

- Increase the wall time: `-W 480` (for 8 hours)
- Optimize your code or reduce input size
- Break the job into smaller tasks
- Use checkpointing if your program supports it

4. Wrong Type (WT) - Incorrect Resource Specification

Error message:

```
Job not started: requested resource [avx512] not available
```

Cause: You requested a resource (CPU instruction set, GPU, specific node type) that doesn't exist or isn't available.

Solutions:

- Check available resources at the [cluster status page](#)
- Remove or modify the resource specification
- Use more general resource requests

Common resource specifications on NCSU HPC:

```
#BSUB -R "select[avx2]"      # AVX2 instruction set
#BSUB -R "select[qc]"       # 8-core nodes
#BSUB -R "select[sc]"       # 16-core nodes
#BSUB -R "select[dc]"       # 32-core nodes
#BSUB -q gpu                # GPU queue
```

5. Module Not Found

Error message in stderr:

```
module: command not found
```

or

```
Lmod has detected the following error: The following module(s) are unknown: "blast"
```

Causes:

- Module system not initialized (rarely on NCSU HPC)
- Module name is incorrect or doesn't exist
- Module is not available on the compute nodes

Solutions:

- Check available modules: `module avail`
- Search for the module: `module spider blast`
- Verify correct module name and version
- Load prerequisite modules first

6. Permission Denied

Error message in stderr:

```
./my_script.sh: Permission denied
```

Cause: The script or executable doesn't have execute permissions.

Solution: Make the file executable before submitting:

```
chmod +x my_script.sh
```

Debugging Tips

1. **Check your error file first:** Most problems will show up in the stderr file

```
cat error.JOBID
tail -50 error.JOBID # Last 50 lines often contain the error
```

2. **Test your script interactively first:** Request an interactive session and run commands manually

```
bsub -Is -n 1 -W 30 bash
# Test your commands here before submitting a batch job
```

3. **Use verbose/debug flags:** Many programs have options for detailed output

```
my_program --verbose input.txt
```

4. Add checks in your script:

```
#!/bin/bash
#BSUB -n 1
#BSUB -W 30
#BSUB -J mytest
#BSUB -o output.%J
#BSUB -e error.%J

# Exit on any error
set -e

# Print commands as they execute
set -x

# Check if input file exists
if [ ! -f "input.txt" ]; then
    echo "ERROR: input.txt not found!"
    exit 1
fi

# Your analysis here
my_program input.txt
```

5. Monitor your job: Check on running jobs regularly

```
bjobs -l JOBID # Detailed job info
```
